TEC | Tecnológico de Costa Rica

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computación

Bases de Datos 1

Análisis de Resultados

Segunda Tarea Programada - Cuenta de Ahorros personal y subcuenta objetivo

Profesor: Franco Quirós Ramírez

Integrantes:

Maicol Araya Urbina - 2025088364

Sebastián Alvarado - 2023180057

Fecha de entrega: 12/10/2026

TEC | Tecnológico de Costa Rica

Contenido

Capítulo 1. Introducción 3
Capítulo 2. Ambiente de desarrollo y arquitectura de la aplicación 3
2.1 Ambiente de desarrollo 3
2.2 Arquitectura de la aplicación 5
Capítulo 3. Resultados del proyecto 6
Capítulo 4. Métricas del proyecto 8

Índice de tablas e ilustraciones

Índice de ilustraciones

Ilustración 1. Diagrama del ambiente de desarrollo colaborativo. 4
Ilustración 2. Diagrama de la arquitectura de la aplicación. 6
Ilustración 3. Actividad de commits. 11

Índice de tablas

Tabla 1. Estado de avance de los elementos calificables del proyecto. 8
Tabla 2. Métricas del proyecto. 10

Capítulo 1. Introducción

Este documento presenta el análisis de resultados de la Segunda Tarea Programada del curso Bases de Datos 1, correspondiente al desarrollo de un sistema de cuenta de ahorros personal y subcuenta objetivo. En esta etapa se implementa la base de datos física y un sitio web para el cliente, conectado a ella, que permite dar mantenimiento a los beneficiarios y consultar los estados de cuenta.

El objetivo del proyecto es implementar una base de datos física en SQL Server y la capa lógica de un sitio web para el cliente, conectadas mediante procedimientos almacenados. El sitio permite iniciar sesión, administrar hasta tres beneficiarios por cuenta (con eliminación lógica y una alerta cuando sus porcentajes no suman 100) y consultar los estados de cuenta. Además, una bitácora registra los hechos relevantes, y los catálogos y datos de prueba se cargan desde archivos XML.

El presente documento se organiza en cuatro capítulos. El Capítulo 2 describe el ambiente de desarrollo utilizado y la arquitectura de la aplicación, incluyendo los diagramas correspondientes. El Capítulo 3 presenta los resultados obtenidos para cada elemento evaluable del proyecto, según la rúbrica proporcionada por el curso. El Capítulo 4 resume las métricas cuantitativas recopiladas durante el desarrollo. Este documento se encuentra en construcción y será actualizado conforme avance el proyecto hasta su entrega final.

Capítulo 2. Ambiente de desarrollo y arquitectura de la aplicación

En este capítulo se describe el ambiente de desarrollo colaborativo utilizado por el equipo de trabajo, así como la arquitectura por capas propuesta para la aplicación web. Ambos elementos se ilustran mediante diagramas, según se detalla a continuación.

2.1 Ambiente de desarrollo

El sistema se desarrolló en un equipo con Windows, utilizando Visual Studio Code como editor y Python 3.12 como lenguaje de programación. La aplicación web utiliza Flask, plantillas Jinja2, HTML y CSS. Git permite llevar el control de versiones y GitHub almacena el repositorio del proyecto. SQL Server Management Studio (SSMS) se utiliza para

administrar la base de datos, crear tablas y procedimientos almacenados y ejecutar consultas de prueba. La aplicación se publica en Vercel a partir del repositorio de GitHub. Los usuarios acceden desde un navegador mediante HTTPS. La aplicación se conecta con Azure SQL Database, donde se aloja la base TP2-BaseDatos del servidor lógico tp2-basedatos. Las conexiones desde Flask utilizan el driver python-tds, el protocolo TDS y cifrado TLS por el puerto TCP 1433. Las pruebas locales de Flask acceden al mismo servicio de base de datos en la nube. El acceso público al servidor está regulado mediante reglas de firewall y autenticación SQL. La configuración de conexión y la clave de sesión se almacenan en variables de entorno.

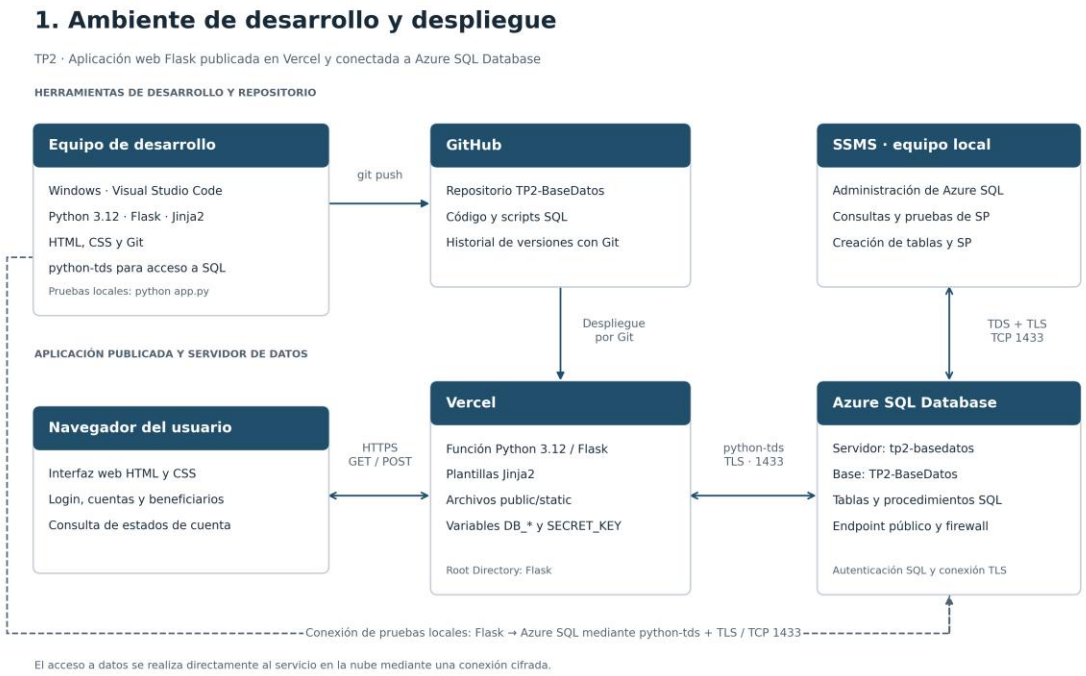


Ilustración 1. Diagrama del ambiente de desarrollo colaborativo.

La Ilustración 1 muestra el ambiente de desarrollo: el código se escribe en Visual Studio Code, se sube a GitHub y se despliega en Vercel, que se conecta a Azure SQL Database, la cual también se administra desde SSMS.

2.2 Arquitectura de la aplicación

El sistema utiliza una arquitectura web cliente-servidor con responsabilidades organizadas de forma similar al patrón Modelo-Vista-Controlador (MVC). Las vistas son las plantillas Jinja2 de Flask/Templates, junto con los estilos CSS que se publican desde public/static. Estas vistas presentan el formulario de ingreso, las cuentas, los beneficiarios y los estados de cuenta. Los controladores son las rutas y funciones de app.py. Reciben solicitudes GET y POST, procesan formularios, validan los datos, verifican las sesiones y los permisos, coordinan las consultas a la base de datos y seleccionan la vista o redirección correspondiente. Las responsabilidades equivalentes al modelo se apoyan en las tablas y los procedimientos almacenados de Azure SQL Database. Las funciones obtener_conexion, ejecutar_sp y consultar_sp, también definidas en app.py, forman el acceso a datos mediante python-tds y parámetros SQL. Los procedimientos consultan o modifican la información y aplican reglas de negocio; los resultados regresan a los controladores para su presentación. La separación es funcional: las vistas se encuentran en archivos independientes, mientras que los controladores y las funciones de acceso a datos comparten app.py. Por ello, la organización se describe como similar a MVC, sin atribuirle una separación física completa en módulos Modelo, Vista y Controlador.

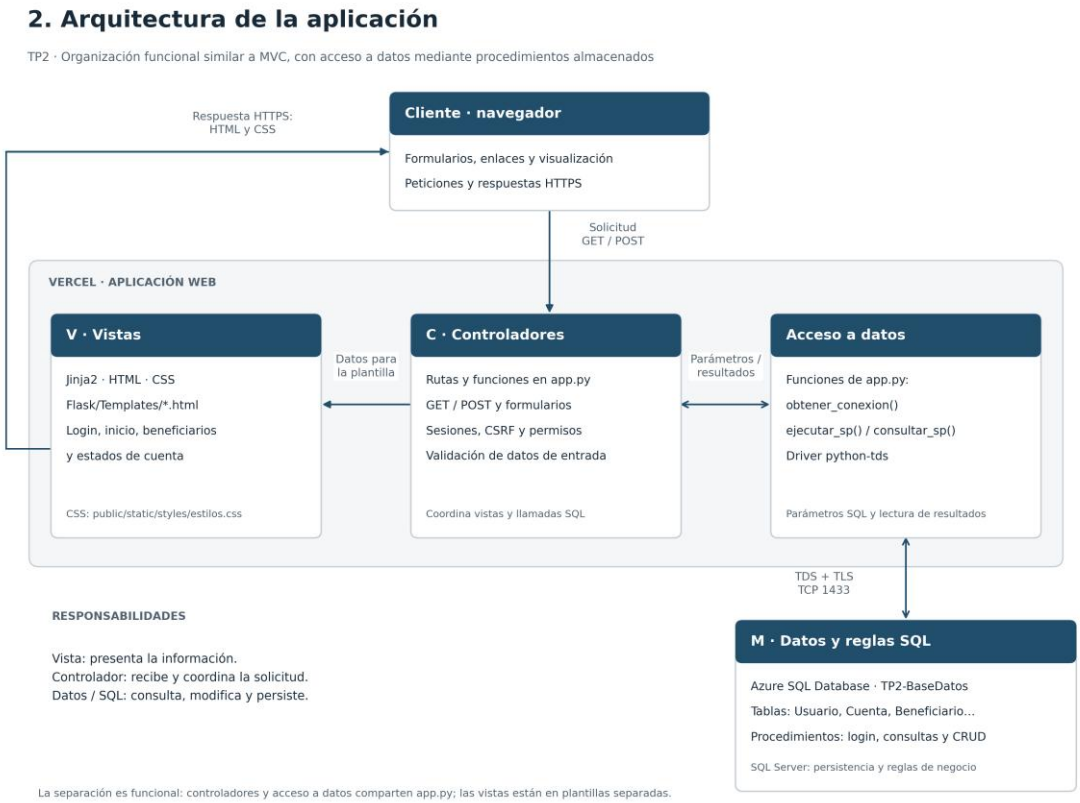


Ilustración 2. Diagrama de la arquitectura de la aplicación.

La Ilustración 2 muestra la arquitectura de la aplicación: el navegador envía solicitudes a los controladores de app.py, que usan los procedimientos almacenados de Azure SQL y entregan los datos a las vistas Jinja2 para responder al usuario.

Capítulo 3. Resultados del proyecto

Este capítulo presenta el estado final de cada uno de los elementos calificables de la Segunda Tarea Programada. El objetivo es ofrecer una visión clara y honesta del grado de cumplimiento alcanzado en cada requerimiento: documentación, diseño de base de datos, carga de datos de prueba, procedimientos almacenados, bitácora, conexión desde la capa lógica y funcionalidades del sitio web, indicando en cada caso el porcentaje de avance y un breve comentario.

Ítem	Resultado	Comentario
Documentación — Bitácora	100%	Se registran entradas en Blogger por cada sesión de trabajo, desde el diseño del modelo físico hasta la conexión del sitio web, con el relato de los problemas encontrados y su solución.
Documentación — Análisis de resultados	100%	Documento completo con portada, índice de contenido, índice de figuras, introducción, diagramas del ambiente de desarrollo y de la arquitectura de la aplicación, tabla de resultados por elemento evaluable y tabla de métricas del proyecto.
Creación de base de datos	100%	Base de datos Tarea1BD y tabla Empleado creadas conforme al modelo físico indicado en el enunciado (id INT IDENTITY PK, Nombre VARCHAR (128) NOT NULL, Salario MONEY NOT NULL).
Datos de prueba	100%	Procedimiento almacenado sp_CargarDatosPrueba que lee un XML con .nodes() y .value().
Procedimientos almacenados (SP)	100%	Se crearon 13 SP: validación de login y logout, consulta de cuentas del usuario, consulta de parentescos, inserción, actualización, actualización de porcentaje y eliminación lógica de beneficiarios, verificación de porcentajes, consulta de estados de cuenta, registro en bitácora y carga de datos.
Bitácora de operaciones en la base de datos	100%	Registra login, logout, modificaciones de beneficiarios (Con JSON de los datos antes y después del cambio) y consultas de estados de cuenta, con fecha y hora.
Conexión a la base de datos desde capa lógica	100%	La aplicación Flask se conecta a Azure SQL Database mediante pyodbc y solo invoca procedimientos almacenados, sin sentencias SQL de negocio embebidas. Lee los result sets y los códigos de retorno de cada SP.

Inicio y cierre de sesión	100%	El formulario de inserción valida los campos en la capa de presentación (nombre y formato de salario), invoca sp_InsertarEmpleado, y muestra el mensaje de éxito o el de "Nombre de Empleado ya existe" según corresponda, refrescando el grid tras una inserción exitosa.
Edición de beneficiarios	100%	Permite agregar (máximo 3 activos por cuenta), actualizar nombre, parentesco, porcentaje y demás datos, y eliminar de forma lógica.
Alerta de porcentajes de beneficiarios	100%	Si la suma de los porcentajes de los beneficiarios activos no es 100, se muestra una alerta visible con el texto indicado en el enunciado.
Consulta de estados de cuenta	100%	Lista los últimos 8 estados de cuenta, del más reciente al más antiguo, con fecha, saldo inicial, saldo final, saldo mínimo, intereses acumulados, cantidad de retiros y depósitos, y transferencias SINPE entrantes y salientes.

Tabla 1. Estado de avance de los elementos calificables del proyecto.

La Tabla 1 presenta el estado de avance de cada uno de los elementos calificables del proyecto. Para cada ítem se indica un porcentaje de realización y un comentario que explica el estado actual y, en los casos incompletos, lo que falta para alcanzar el 100%.

Capítulo 4. Métricas del proyecto

Este capítulo presenta un conjunto de métricas cuantitativas recopiladas a lo largo del desarrollo del proyecto, con el propósito de dimensionar objetivamente el esfuerzo invertido y la evolución del trabajo en el tiempo.

Métrica	Valor	Comentario
Fecha de inicio del proyecto	21 de septiembre de 2026	Fecha de creación del repositorio en GitHub

Fecha de la primera entrada en GitHub	21 de septiembre de 2026	Initial commit
Total de horas-Maicol Araya Urbina	20.5 horas	Se obtuvieron de las entradas del blog
Total de horas- Sebastián Alvarado	21 horas	Se obtuvieron de las entradas del blog
Cantidad de entradas en el blog	11 entradas	Repartidas entre el 21 de septiembre y 12 de octubre
Cantidad de entradas (commits) en GitHub	25	Repartidas entre el 21 de septiembre y 12 de octubre
Fecha de finalización del proyecto	12 de octubre de 2026	Fecha en la que se subió este documento al repositorio
Cantidad de tablas creadas	12	TipoMoneda, TipoDocuIdentidad, Parentezco, TipoCuentaAhorro, TipoOperaciones, Persona, Cuenta, Beneficiario, EstadoDeCuenta, Usuario, UsuarioCuenta, Bitacora
Cantidad de procedimientos almacenados	15	sp_RegistrarBitacora, sp_ObtenerParentescos, sp_ValidarLogin, sp_LogoutUsuario, sp_ObtenerCuentasUsuario, sp_ObtenerBeneficiarios, sp_VerificarPorcentajeBeneficiarios, sp_ObtenerEstadosCuenta, sp_InsertarBeneficiario, sp_ActualizarBeneficiario, sp_ActualizarPorcentajeBeneficiario, sp_EliminarBeneficiario, sp_ObtenerTiposDocumento, sp_ObtenerBeneficiarioPorId, sp_CargarDatosPrueba

Cantidad de funciones	0	No se crearon funciones adicionales, solo procedimientos almacenados.
Cantidad de datos procesados en prueba	636 filas	27 de catálogos y 609 de entidades (60 personas, 25 cuentas, 51 beneficiarios, 424 estados de cuenta, 21 usuarios y 28 asociaciones usuario-cuenta), cargadas desde un archivo XML.
Cantidad de pruebas realizadas	15	Login y logout, agregar, actualizar, cambiar porcentaje y eliminar beneficiarios, consulta de estados de cuenta, alerta de porcentajes y casos de error.
Tiempo de ejecución del script de carga de datos	4 segundos	Se puede medir en SSMS.
Cantidad de líneas de código (capa lógica, Python)	390	líneas de app.py
Cantidad de líneas de código SQL	1100	Cantidad de líneas sumadas de los scripts .sql del proyecto.

Tabla 2. Métricas del proyecto.

La Tabla 2 resume las principales métricas cuantitativas recopiladas durante el desarrollo del proyecto hasta el momento, separando las horas trabajadas por cada integrante del equipo.

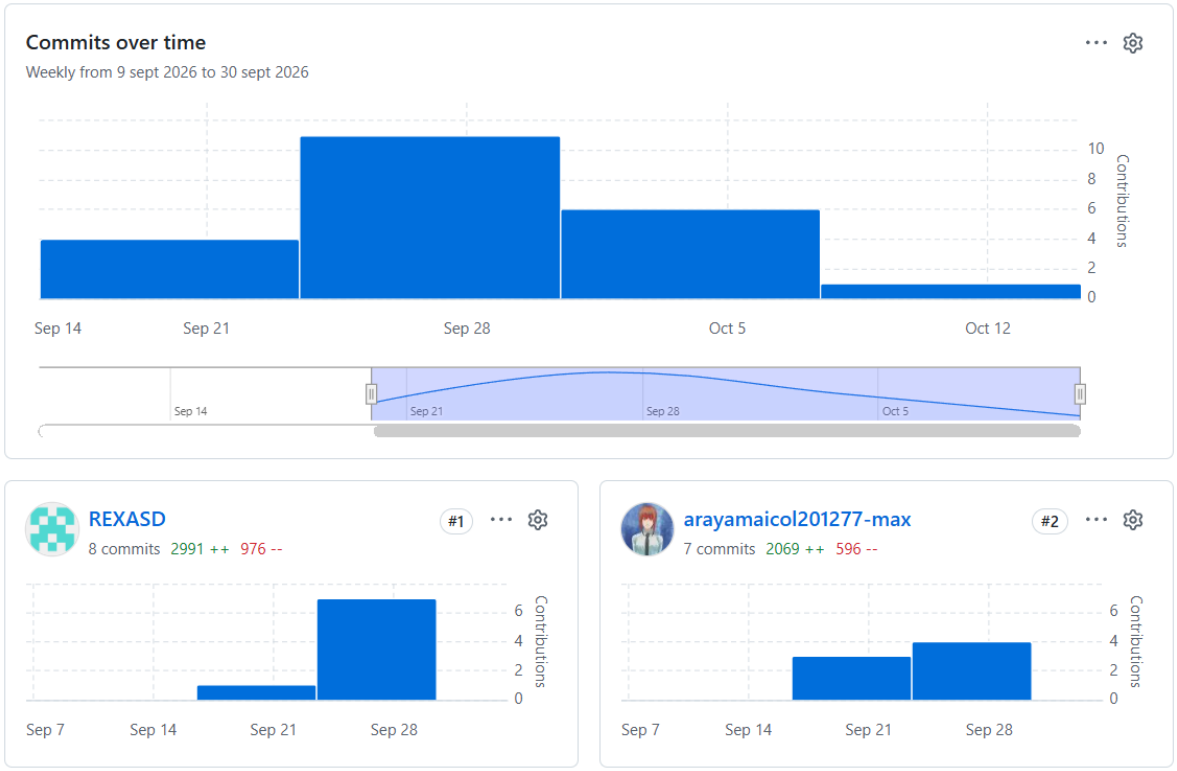


Ilustración 3. Actividad de commits.

Como evidencia adicional de trabajo distribuido y colaborativo a lo largo del proyecto, la Ilustración 3 muestra la actividad de commits realizada, obtenida directamente de la sección Insights del repositorio en GitHub.